Integrated Systems Laboratory
Institut für Integrierte Systeme

Department of Information Technology and Electrical Engineering
Spring Semester 26

A Comprehensive Study of Integrated Systems

Master Thesis

Student Name
student@iis.ee.ethz.ch

March 17, 2026

Advisors:

- First Supervisor, first.supervisor@iis.ee.ethz.ch
- Second Supervisor, second.supervisor@iis.ee.ethz.ch

Professor:

- Prof. Dr. P. Professor, professor@iis.ee.ethz.ch

Acknowledgements

I would like to thank my supervisors for their guidance and support throughout this project. Special thanks to the Integrated Systems Laboratory for providing the resources necessary to complete this work.

Abstract

This thesis presents a comprehensive study of integrated systems design at ETH Zurich's Integrated Systems Laboratory. We explore novel approaches to hardware architecture, implementation, and verification. Our results demonstrate significant improvements over the state of the art in terms of performance and energy efficiency.

Declaration of Originality

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor. For a detailed version of the declaration of originality, please refer to Section C.

List of Acronyms

ASIC	Application-Specific Integrated Circuit
FPGA	Field-Programmable Gate Array
HDL	Hardware Description Language
IC	Integrated Circuit
IIS	Integrated Systems Laboratory
RTL	Register-Transfer Level
SoC	System-on-Chip

Contents

1 Introduction **1**
1.1 Motivation 1
1.2 Contributions 1
1.3 Outline 1
2 Background **2**
2.1 Integrated Circuits 2
2.2 Hardware Description Languages 2
3 Related Work **3**
4 Theory **4**
4.1 Problem Formulation 4
4.2 Algorithmic Approach 4
5 Architecture **5**
5.1 Overview 5
5.2 Frontend 5
5.3 Backend 5
6 Implementation **6**
6.1 RTL Design 6
6.2 Synthesis Results 6
7 Results **7**
7.1 Experimental Setup 7
7.2 Performance 7
8 Conclusion **8**
8.1 Future Work 8
A Typst Quick Guide **9**
A.1 Document Structure 9
A.2 Text Formatting 9
A.3 Mathematics 10
A.4 Figures 10
A.5 Tables 11
A.6 Labels and Cross-References 11
A.7 Bibliography and Citations 11
A.8 Lists 12
A.9 Including External Files 12
A.10 Useful Packages 12
A.11 Appendices 12
B Assignment Description **14**
C Declaration of Originality **15**
Bibliography **16**

List of Figures

Figure 1 High-level architecture of the proposed design. 5

List of Tables

Table 1 Post-synthesis results on 22 nm technology. 6

1

Introduction

Modern integrated systems face increasing demands for performance, energy efficiency, and reliability. This thesis addresses these challenges by proposing a novel architecture that leverages recent advances in hardware design.

1.1 Motivation

The growing complexity of Integrated Circuits (ICs) requires new design methodologies that can handle billions of transistors while maintaining correctness and meeting strict timing constraints.

1.2 Contributions

The main contributions of this thesis are:

- A novel hardware architecture for efficient data processing
- A verification methodology that scales to large designs
- A comprehensive evaluation on real silicon

1.3 Outline

The remainder of this thesis is organized as follows. Section 2 reviews relevant background material. Section 3 discusses related work. Section 4 presents the theoretical foundations. Section 5 describes the proposed architecture. Section 6 details the implementation. Section 7 presents the evaluation results. Section 8 concludes the thesis.

2

Background

This chapter introduces the background knowledge required to understand this thesis.

2.1 Integrated Circuits

An IC is a set of electronic circuits on a small flat piece of semiconductor material. Modern ICs contain billions of transistors and operate at clock frequencies exceeding several gigahertz. Systems-on-Chip (SoCs) integrate a complete system — processor, memory, and peripherals — onto a single die, and are the focus of research at the Integrated Systems Laboratory (IIS).

2.2 Hardware Description Languages

Hardware Description Languages (HDLs) are used to describe the structure and behavior of electronic circuits. The most commonly used HDLs are VHDL and SystemVerilog. Designs are typically written at the Register-Transfer Level (RTL) abstraction and then synthesized to gates for an Application-Specific Integrated Circuit (ASIC) or mapped to an Field-Programmable Gate Array (FPGA).

3

Related Work

This chapter reviews existing work in the field and positions our contributions relative to the state of the art.

Several groups have proposed similar architectures. However, none of these approaches achieve the combination of performance and energy efficiency presented in this work.

4

Theory

This chapter presents the theoretical foundations underlying our approach.

4.1 Problem Formulation

Let $G = (V, E)$ be a directed graph representing the dataflow of a computation, where V is the set of operations and E is the set of data dependencies between them.

4.2 Algorithmic Approach

Our approach builds on the following key insight: by exploiting the structure of G, we can schedule operations more efficiently than existing methods.

5

Architecture

This chapter describes the proposed hardware architecture in detail.

5.1 Overview

Figure 1 shows the high-level block diagram of the proposed architecture. The design consists of three main components: a frontend, a backend, and a memory subsystem.

Figure 1: High-level architecture of the proposed design.

5.2 Frontend

The frontend is responsible for fetching and decoding instructions. It implements a speculative execution pipeline with branch prediction.

5.3 Backend

The backend executes decoded instructions out of order, exploiting instruction-level parallelism.

6

Implementation

This chapter describes the RTL implementation and the physical design flow.

6.1 RTL Design

The design is implemented in SystemVerilog following the lowRISC coding style guide. The top-level module instantiates the frontend, backend, and memory subsystem.

6.2 Synthesis Results

Table 1 summarizes the synthesis results for the proposed design on a 22 nm technology node.

Module	Area [kGE]	Freq. [MHz]
Frontend	42.3	800
Backend	128.7	800
Memory	35.1	800
Total	**206.1**	**800**

Table 1: Post-synthesis results on 22 nm technology.

7

Results

This chapter presents the experimental evaluation of the proposed design.

7.1 Experimental Setup

We evaluate the design using a suite of standard benchmarks. All measurements are performed on post-layout netlist simulations.

7.2 Performance

Our design achieves an average speedup of 1.8× over the baseline architecture while consuming 30% less energy.

8

Conclusion

This thesis presented a novel integrated systems architecture that achieves significant improvements in performance and energy efficiency. The key insight was to exploit dataflow structure to improve scheduling.

8.1 Future Work

Future work will explore extending the architecture to support multi-core configurations and investigate further optimizations at the physical design level.

A

Typst Quick Guide

Typst is a modern typesetting system designed as a faster, more ergonomic alternative to LaTeX. This guide covers the most common tasks for writing a thesis, with LaTeX equivalents shown alongside every Typst example.

> **Info**
>
> The full Typst documentation is available at https://typst.app/docs. The interactive web app at https://typst.app lets you experiment without any local installation.

A.1 Document Structure

A thesis file using this template starts with:

Code

```
#import "/templates/thesis.typ": *
#import "/acronyms.typ": acronyms

#show: thesis.with(
  title: "My Thesis",
  author: "Jane Doe",
  // ... other parameters
)

= Introduction    // chapter
== Background     // section
=== Details       // subsection
```

In LaTeX this corresponds to `\chapter{}`, `\section{}`, `\subsection{}`. There is no preamble or `\begin{document}` — the template handles all of that.

A.2 Text Formatting

Feature	LaTeX	Typst
Bold	`\textbf{text}`	`*text*` or `#strong[text]`
Italic	`\textit{text}`	`_text_` or `#emph[text]`
Monospace	`\texttt{text}`	`` `text` ``
Superscript	`x\textsuperscript{2}`	`x#super[2]`

Feature	LaTeX	Typst
Subscript	`x\textsubscript{i}`	`x#sub[i]`
Forced linebreak	`\\`	`\` (backslash + space)
Non-breaking space	`~`	`~`
Quotation marks	``` ``text'' ```	`"text"` (auto smart quotes)
Hyperlink	`\href{url}{label}`	`#link("url")[label]`

A.3 Mathematics

Typst uses `$...$` for both inline and display math. Adding spaces *inside* the delimiters switches to display (block) mode:

Feature	LaTeX	Typst
Inline math	`$x^2$`	`$x^2$`
Display math	`\[ x^2 \]`	`$ x^2 $` (spaces inside `$`)
Fraction	`\frac{a}{b}`	`$a/b$`
Square root	`\sqrt{x}`	`$sqrt(x)$`
Sum / integral	`\sum_{i=0}^{n}`	`$sum_(i=0)^n$`
Greek letters	`\alpha, \beta`	`$alpha, beta$`
Bold math	`\mathbf{A}`	`$bold(A)$`
Aligned equations	`\begin{align}.. \end{align}`	`$ a &= b \ c &= d $`

🔥 **Tip**

Math function names in Typst have no backslash: `sin`, `cos`, `lim`, `max`, etc. See the symbol reference for the full list of available symbols.

A.4 Figures

</> **Code**

```
#figure(
  image("figures/arch.svg", width: 80%),
  caption: [A descriptive caption.],
) <fig:arch>

// Reference it anywhere in the text:
As shown in @fig:arch ...
```

Labels are placed with `<label>` directly after the closing `)` of `figure()`. `@label` is the equivalent of `\autoref{}` — Typst automatically prefixes “Figure”, “Table”, “Section”, etc.

A.5 Tables

Typst tables use `stroke: none` with `table.hline` for booktabs-style rules:

</> Code

```
#figure(
  table(
    columns: (auto, 1fr, 1fr),
    stroke: none,
    table.hline(stroke: 1.5pt),                // \toprule
    table.header([*Config*], [*Freq.*], [*Area*]),
    table.hline(stroke: 0.75pt),               // \midrule
    [Baseline], [500 MHz], [142 500 µm²],
    [Opt.],     [700 MHz], [164 900 µm²],
    table.hline(stroke: 1.5pt),                // \bottomrule
  ),
  caption: [Synthesis results.],
) <tab:results>
```

Column widths accept `auto` (fit content), fixed lengths (`3cm`), or fractional (`1fr`, `2fr`) to share available space proportionally.

A.6 Labels and Cross-References

</> Code

```
= Introduction <chap:intro>     // label a heading
#figure(...) <fig:arch>         // label a figure

As shown in @fig:arch and discussed in @chap:intro ...
```

A.7 Bibliography and Citations

</> Code

```
#show: thesis.with(
  bibliography: bibliography("references.bib", style: "ieee", full:
true),
)

As shown in @smith2023 ...      // equivalent to \cite{smith2023}
```

The `.bib` format is identical to LaTeX. The bibliography is rendered automatically at the end of the document.

A.8 Lists

Code

```
// Unordered list (\begin{itemize})
- First item
- Second item
  - Nested item

// Ordered list (\begin{enumerate})
+ First item
+ Second item

// Term list (\begin{description})
/ Term: Definition of the term.
```

A.9 Including External Files

Code

```
// Render content from another file (\input equivalent):
include "/chapters/introduction.typ"

// Import bindings without rendering (for macros / dicts):
#import "/acronyms.typ": acronyms
```

A.10 Useful Packages

Package	Purpose
`@preview/cetz`	Drawing and diagrams (TikZ equivalent)
`@preview/acrostiche`	Acronym management (`\newacronym` equivalent)
`@preview/gentle-clues`	Callout / admonition boxes
`@preview/lovelace`	Algorithm pseudocode
`@preview/equate`	Numbered sub-equations

Packages are imported with a version-pinned line — Typst downloads them automatically on first use, no manual installation needed:

Code

```
#import "@preview/cetz:0.4.2": canvas, draw
```

Browse the full package registry at https://typst.app/universe.

A.11 Appendices

This guide is included as the default appendix. Once you add your own `appendices` parameter, the guide is replaced by your content:

Code

```
#show: thesis.with(
  appendices: (
    (title: "Supplementary Data", body: [Your content here.]),
    (title: "Source Code",        body: include "/chapters/
appendix_code.typ"),
  ),
)
```

Each entry is a dict with a `title` string and a `body` content block. To include this guide alongside your own appendices, add it explicitly:

Code

```
appendices: (
  (title: "My Appendix",       body: include "/chapters/
appendix.typ"),
  (title: "Typst Quick Guide", body: include "/templates/typst-
guide.typ"),
),
```

To omit all appendices, pass an empty array:

Code

```
appendices: (),
```

B

Assignment Description

Add Assignment Description

Include the assignment description PDF you received from your advisor:

Code

```
#show: thesis.with(
  assignment-description: include-pdf("/figures/assignment.pdf",
pages: 3),
  // ...
)
```

C

Declaration of Originality

Add Declaration of Originality

Download, sign, and scan the official ETH Zurich declaration of originality, then include it here. The form is available at:

- English version
- German version

Code

```
#show: thesis.with(
  declaration-of-originality: include-pdf("/figures/
declaration_of_originality.pdf"),
  // ...
)
```

Bibliography

[1] L. Mädje, "A Programmable Markup Language for Typesetting," *Typst Research*, 2022.

[2] M. E. Haug, "Fast typesetting with incremental compilation," *Master's thesis, Technical University of Berlin, Germany*, 2022.